

# $\pi$ Tantum

alias *Tempus Tantum*

*sistema di generazione orario scolastico*

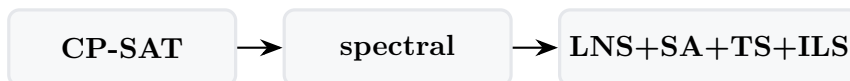
---

*“Omnia, Lucili, aliena sunt, tempus tantum nostrum est.”*  
Seneca, *Epistulae morales ad Lucilium*, I, 1

*“Tutto, Lucilio mio, ci appartiene di altri; soltanto il tempo e’ nostro.”*

---

## Manuale tecnico



Giovanni Borghi

3 maggio 2026

# Indice

|          |                                                |           |
|----------|------------------------------------------------|-----------|
| <b>1</b> | <b>Introduzione</b>                            | <b>3</b>  |
| 1.1      | Avvio rapido . . . . .                         | 3         |
| 1.2      | Stato attuale del progetto . . . . .           | 3         |
| <b>2</b> | <b>Installazione</b>                           | <b>5</b>  |
| 2.1      | Windows . . . . .                              | 5         |
| 2.2      | Linux (Ubuntu, Debian, Fedora, Arch) . . . . . | 6         |
| 2.3      | macOS (Intel + Apple Silicon) . . . . .        | 7         |
| 2.4      | Verifica installazione . . . . .               | 7         |
| 2.5      | Aggiornamento . . . . .                        | 8         |
| 2.6      | Disinstallazione . . . . .                     | 8         |
| <b>3</b> | <b>Architettura</b>                            | <b>9</b>  |
| 3.1      | Stack . . . . .                                | 9         |
| 3.2      | Diagramma a blocchi . . . . .                  | 9         |
| 3.3      | Struttura cartelle . . . . .                   | 9         |
| 3.4      | Lifecycle del backend . . . . .                | 10        |
| <b>4</b> | <b>Modello dati</b>                            | <b>11</b> |
| 4.1      | Famiglie di tabelle . . . . .                  | 11        |
| 4.2      | Diagramma relazioni . . . . .                  | 11        |
| 4.3      | Migrazioni idempotenti . . . . .               | 11        |
| 4.4      | Pickle dell'engine . . . . .                   | 12        |
| <b>5</b> | <b>Vincoli</b>                                 | <b>13</b> |
| 5.1      | I 5 stati / 4 kind . . . . .                   | 13        |
| 5.2      | Matrici di disponibilità 5-stati . . . . .     | 13        |
| 5.3      | Vincoli logici (DNF) . . . . .                 | 13        |
| 5.3.1    | Sintassi . . . . .                             | 13        |
| 5.3.2    | Predicate atoms . . . . .                      | 14        |
| 5.3.3    | Esempi pratici . . . . .                       | 14        |
| 5.3.4    | Mapping CP-SAT . . . . .                       | 14        |
| 5.4      | Vincoli HARD a toggle (per classe) . . . . .   | 15        |
| 5.5      | Conflict detection . . . . .                   | 15        |
| <b>6</b> | <b>Workflow di ottimizzazione</b>              | <b>16</b> |
| 6.1      | Schema delle fasi . . . . .                    | 16        |
| 6.2      | Phase A: assegnazione (cattedre) . . . . .     | 16        |
| 6.3      | Phase B: scheduling . . . . .                  | 16        |
| 6.4      | Metaeuristiche . . . . .                       | 16        |
| 6.5      | Phase 4: assegnazione aule . . . . .           | 17        |
| 6.6      | Drag-and-drop con preview live . . . . .       | 17        |

|           |                                                      |           |
|-----------|------------------------------------------------------|-----------|
| 6.6.1     | Room-follows-lesson . . . . .                        | 17        |
| 6.7       | Slot picker matriciale (Monitor) . . . . .           | 18        |
| <b>7</b>  | <b>Guida UI</b>                                      | <b>19</b> |
| 7.1       | Funzionalità trasversali . . . . .                   | 19        |
| 7.1.1     | Query DSL e sort multi-livello . . . . .             | 19        |
| 7.1.2     | Multi-select . . . . .                               | 19        |
| 7.1.3     | Excel/CSV import . . . . .                           | 19        |
| 7.1.4     | Esportazione liste (xlsx / csv) . . . . .            | 19        |
| 7.1.5     | Viste salvate . . . . .                              | 20        |
| 7.1.6     | Tag (aule e studenti) . . . . .                      | 20        |
| 7.1.7     | Import bulk dedicato (/import) . . . . .             | 20        |
| 7.1.8     | Import / Export DB (Dashboard) . . . . .             | 20        |
| 7.2       | Tab per tab . . . . .                                | 21        |
| <b>8</b>  | <b>Tecniche di ottimizzazione avanzate</b>           | <b>22</b> |
| 8.1       | Adaptive Large Neighborhood Search (ALNS) . . . . .  | 22        |
| 8.2       | Variable Neighborhood Search (VNS) . . . . .         | 22        |
| 8.3       | Hall's theorem pre-check . . . . .                   | 22        |
| 8.4       | Column Generation (Dantzig-Wolfe) . . . . .          | 22        |
| 8.5       | Lagrangian Relaxation . . . . .                      | 22        |
| <b>9</b>  | <b>DSL generico per i vincoli</b>                    | <b>23</b> |
| 9.1       | Filosofia . . . . .                                  | 23        |
| 9.2       | Grammatica BNF . . . . .                             | 23        |
| 9.3       | Sorgenti iterabili . . . . .                         | 24        |
| 9.4       | Galleria di esempi . . . . .                         | 24        |
| 9.4.1     | Vincoli docente (10 esempi) . . . . .                | 24        |
| 9.4.2     | Vincoli classe (5 esempi) . . . . .                  | 25        |
| 9.4.3     | Vincoli aula (5 esempi) . . . . .                    | 26        |
| 9.4.4     | Vincoli materia (3 esempi) . . . . .                 | 27        |
| 9.4.5     | Vincoli gruppo / studenti (3 esempi) . . . . .       | 27        |
| 9.4.6     | Vincoli relazionali / coupling (4 esempi) . . . . .  | 27        |
| 9.5       | Errori comuni . . . . .                              | 28        |
| <b>10</b> | <b>Diagnostica statistica</b>                        | <b>29</b> |
| 10.1      | Sensitivity Monte Carlo . . . . .                    | 29        |
| 10.2      | Analisi bipartito . . . . .                          | 29        |
| 10.3      | Correlazioni e regressioni . . . . .                 | 29        |
| 10.4      | Distribuzioni . . . . .                              | 29        |
| 10.5      | Run telemetry . . . . .                              | 29        |
| <b>11</b> | <b>API REST</b>                                      | <b>30</b> |
| 11.1      | Pattern CRUD . . . . .                               | 30        |
| 11.2      | Esempi curl . . . . .                                | 30        |
| 11.3      | Riferimento gruppi di endpoint . . . . .             | 31        |
| <b>12</b> | <b>Estendere il sistema</b>                          | <b>32</b> |
| 12.1      | Aggiungere una nuova tabella + CRUD . . . . .        | 32        |
| 12.2      | Migrazione idempotente . . . . .                     | 32        |
| 12.3      | Aggiungere un nuovo tipo di vincolo . . . . .        | 32        |
| 12.4      | Aggiungere un nuovo step di ottimizzazione . . . . . | 32        |
| 12.5      | Testing . . . . .                                    | 33        |

---

|                                                                           |           |
|---------------------------------------------------------------------------|-----------|
| <b>A Launcher</b>                                                         | <b>34</b> |
| A.1 Windows: <code>start.bat</code> . . . . .                             | 34        |
| A.2 Linux / macOS: <code>start.sh</code> + <code>stop.sh</code> . . . . . | 34        |
| <b>B Repository GitHub</b>                                                | <b>35</b> |
| <b>C Reference</b>                                                        | <b>36</b> |

# Elenco delle figure

|     |                                                              |    |
|-----|--------------------------------------------------------------|----|
| 3.1 | Architettura a blocchi della webapp. . . . .                 | 9  |
| 4.1 | Relazioni rilevanti del modello dati (semplificato). . . . . | 12 |
| 6.1 | Pipeline a 4 fasi. . . . .                                   | 16 |

# 1. Introduzione

$\pi$  *Tantum* (alias *Tempus Tantum*, codename interno: **timetable**) è un sistema di generazione e gestione dell’orario scolastico per un Liceo italiano. Il nome richiama il celebre incipit dell’epistola I di Seneca a Lucilio: “Omnia, Lucili, aliena sunt, tempus tantum nostrum est” (Seneca, *Epistulae morales ad Lucilium*, I, 1) – “tutto ci appartiene di altri, soltanto il tempo è nostro”. Il gioco grafico fra la lettera greca  $\pi$  e le due “T” di *Tempus Tantum* riassume l’idea: organizzare il tempo della scuola, dato che del tempo non si dispone se non gestendolo.

È composto di tre layer:

1. **Solver** (`experiments/`): pipeline CP-SAT (Google OR-Tools) con decomposizione spettrale per istanze grandi e cascata metaeuristica (LNS, SA, TS, ILS).
2. **Web UI** (`webui/`): backend FastAPI su SQLite + frontend SvelteKit. È il punto di accesso quotidiano: gestisce CRUD, ottimizzazione, visualizzazione, drag-and-drop, assenze e supplenze.
3. **Legacy** (`schedule/`): script monolitici da cui il solver è nato; mantenuti per riferimento storico.

Il manuale copre tutti e tre i layer ma si concentra sulla web UI, che ha assorbito quasi ogni funzionalità operativa.

## 1.1 Avvio rapido

**Windows.** Doppio click su `webui/start.bat`. Apre due finestre cmd: backend *uvicorn* su porta 8000, frontend *vite* su porta 5173.

### Linux / macOS.

```
cd timetable/webui
./start.sh # background; URLs sulla console
./stop.sh # ferma backend + frontend
```

Nel browser, aprire `http://127.0.0.1:5173` e dalla **Dashboard** importare un profilo già calcolato (`small` per cominciare).

## 1.2 Stato attuale del progetto

Le funzionalità principali, al momento di questa redazione:

- 16 sezioni (tab) della UI, ~118 endpoint REST.
- ~30 tabelle SQLAlchemy con migrazioni idempotenti.
- 5 stati di vincoli (HARD/SOFT/PREFERITO/ENFORCED/ALLOWED).

- Vincoli logici DNF con DSL esteso (predicate atoms `aula:`, `materia:`, `classe:`, `gruppo:`, `docente:`).
- Conflict detection automatico nel tab *Vincoli*.
- Drag-and-drop con preview live (rosso/giallo/verde) e *room-follows-lesson*.
- Gestione assenze settimanali con assegnazione supplenti drag-drop.
- Tab *Monitor* con espansione lezione-per-lezione e slot picker matriciale.

## 2. Installazione

Guida cross-platform per installare ed eseguire piTantum su Windows, Linux e macOS. La versione *di riferimento* di questa guida è `docs/installation.md` nel repo (Markdown, sempre aggiornato); qui se ne riproduce il contenuto in forma compatta per il manuale PDF. In caso di divergenza, fa fede il file Markdown.

**Architettura del lancio.** Il progetto è composto da due processi:

- **Backend:** `uvicorn` che serve l'app FastAPI in `webui/backend` sulla porta 8000.
- **Frontend:** server di sviluppo `vite` che serve l'app SvelteKit in `webui/frontend` sulla porta 5173.

Uno script unico (`start.bat` su Windows, `start.sh` su Linux/macOS) li avvia entrambi. Al primo lancio crea la `venv` Python, installa i requirements, fa `npm install`; ai lanci successivi parte in pochi secondi.

### 2.1 Windows

**Prerequisiti.**

- Python  $\geq 3.11$  – installer ufficiale: <https://www.python.org/downloads/windows/>
- Node.js  $\geq 20$  LTS – installer ufficiale: <https://nodejs.org/en/download> (scegliere “LTS”).
- Git – installer ufficiale: <https://git-scm.com/download/win>.

**Note importanti.**

- Sull'installer Python, **spuntare** “Add python.exe to PATH”. Senza, lo script non trova il comando `python`.
- Riavviare il terminale dopo le installazioni: il PATH non si aggiorna nelle finestre già aperte.

**Clone + lancio.**

```
git clone https://github.com/gborghi/timetable.git
cd timetable
webui\start.bat
```

Lo script apre due finestre cmd (“piTantum – backend” e “piTantum – frontend”). Aprire il browser su `http://localhost:5173`.

**Stop.** Chiudere le due finestre cmd, oppure `Ctrl+C` in ciascuna.

## Troubleshooting Windows.

- “python non riconosciuto”: l’installer Python non è stato eseguito con “Add to PATH”. Reinstallare oppure aggiungere C:\Users\tu\AppData\Local\Programs\Python\Python311 al PATH.
- Porta 8000 occupata: `netstat -ano | findstr :8000 + taskkill /PID <pid> /F`.
- Antivirus blocca `start.bat`: aggiungere alle eccezioni di Windows Defender.

## 2.2 Linux (Ubuntu, Debian, Fedora, Arch)

**Prerequisiti.** Python  $\geq 3.11$ , Node  $\geq 20$  LTS, Git, build-essential.

### Ubuntu / Debian.

```
sudo apt update
sudo apt install -y python3 python3-venv python3-pip nodejs npm \
git build-essential
```

I repo apt di Ubuntu 22.04 / Debian 12 hanno Node 12-18, troppo vecchio per SvelteKit 2 (richiede 20+). In quel caso usare **nvm**:

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
source ~/.bashrc
nvm install --lts && nvm use --lts
```

### Fedora.

```
sudo dnf install -y python3 python3-virtualenv nodejs npm git \
gcc make
```

### Arch / Manjaro.

```
sudo pacman -S python python-pip nodejs npm git base-devel
```

### Clone + lancio.

```
git clone https://github.com/gborghi/timetable.git
cd timetable
chmod +x webui/start.sh webui/stop.sh
./webui/start.sh # background; log in webui/logs/
./webui/start.sh --foreground # log live in console (Ctrl+C ferma)
```

### Stop.

```
./webui/stop.sh
```

## Troubleshooting Linux.

- Porta in uso: `sudo lsof -i :8000 + kill <pid>` (eventualmente `kill -9 <pid>`).
- python3 mancante su distro minimal: usare l’eseguibile chiamato `python` (3.x).
- WSL2: `start.sh` funziona normalmente. Aprire il browser dal lato Windows su `http://localhost:5173`; WSL2 inoltra le porte automaticamente.

## 2.3 macOS (Intel + Apple Silicon)

### Prerequisiti.

- Xcode Command Line Tools: `xcode-select -install`.
- Homebrew: vedere <https://brew.sh/>.
- Python  $\geq 3.11$ , Node  $\geq 20$  LTS, Git: tutti via Homebrew.

### Installazione.

```
# Homebrew (se non presente)
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

# Toolchain
brew install python@3.12 node git

# Verifica
python3 --version # >= 3.11
node --version # >= v20
```

### Note Apple Silicon.

- ortools dalla v9.7 spedisce wheel ARM64 nativi: Rosetta non serve.
- Se Python è installato da python.org invece che Homebrew, scegliere l'installer "macOS 64-bit universal2".

**Clone + lancio + stop.** Identico a Linux.

### Troubleshooting macOS.

- Gatekeeper blocca `start.sh`: `xattr -d com.apple.quarantine webui/start.sh webui/stop.sh`.
- ortools senza wheel ARM (versioni vecchie): `pip install --upgrade ortools` dentro la venv del backend.
- brew non in PATH: `eval "$(/opt/homebrew/bin/brew shellenv)"` in `~/.zprofile`.

## 2.4 Verifica installazione

Dopo il primo lancio andato a buon fine:

```
python --version # >= 3.11 (Linux/macOS: spesso python3)
node --version # >= v20
npm --version # >= 10

# Backend health
curl http://127.0.0.1:8000/api/health
# atteso: {"status":"ok","name":"pitantum","version":"0.1.0"}

# Async layer (Section 2.5 P2)
curl http://127.0.0.1:8000/api/health/async
# atteso: {"status":"ok","async":true,"dialect":"sqlite"}

# Frontend
curl -I http://127.0.0.1:5173
# atteso: HTTP/1.1 200 OK
```

## 2.5 Aggiornamento

Quando si fa `git pull` per portare a casa nuovi commit:

```
cd /path/to/timetable
git pull origin main

# se sono cambiate dipendenze Python:
webui/backend/.venv/bin/pip install -r webui/backend/requirements.txt
# (Windows: webui\backend\.venv\Scripts\pip install ...)

# se sono cambiate dipendenze frontend:
cd webui/frontend && npm install && cd ../../

# se ci sono migration DB:
cd webui/backend && ../.venv/bin/alembic upgrade head && cd ../../
# (Windows: webui\backend\.venv\Scripts\alembic upgrade head)
```

In pratica `start.sh/start.bat` rilanciato dopo un `git pull` gestisce automaticamente `pip install` e `npm install` quando servono. Le migration DB sono coperte da `_apply_lightweight_migrations()` allo startup come `safety-net` (vedi capitolo *Modello dati*); per la via canonica usare `alembic upgrade head`.

Dopo un aggiornamento backend, fare un **hard reload** del browser (`Ctrl+Shift+R`) per pulire chunk JavaScript cached.

## 2.6 Disinstallazione

```
cd /path/to/timetable
./webui/stop.sh # Linux/macOS
# (Windows: chiudere le finestre cmd di start.bat)

rm -rf webui/backend/.venv # ~150 MB
rm -rf webui/frontend/node_modules # ~400 MB
rm -rf webui/data/timetable.db # DATI UTENTE (salvarli prima!)
```

Per rimuovere completamente, cancellare la cartella `timetable/`. Python, Node, Git restano installati nel sistema; rimuoverli dal gestore di pacchetti solo se non servono ad altri progetti.

## 3. Architettura

### 3.1 Stack

- **Backend:** Python 3.11+, FastAPI, SQLAlchemy 2.0, Pydantic 2, uvicorn, OR-Tools, NumPy, scikit-learn, Faker.
- **Frontend:** SvelteKit (Svelte 4), Vite 5, Tailwind CSS, JavaScript puro.
- **DB:** SQLite via SQLAlchemy. Migrazioni idempotenti nel codice (no Alembic).
- **Engine I/O:** pickle Python; `engine_io.py` converte fra DB e dict per il solver.

### 3.2 Diagramma a blocchi

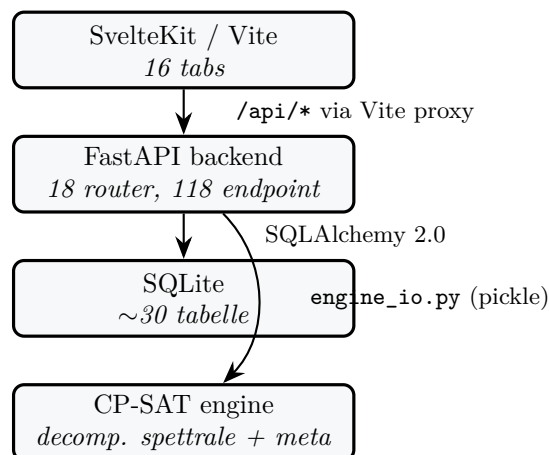


Figura 3.1: Architettura a blocchi della webapp.

### 3.3 Struttura cartelle

```
timetable/  
+-- README.md -- landing page  
+-- docs/ -- manuale tecnico  
+-- experiments/ -- solver code + pickle snapshots  
| +-- cpsat_v2_assignment.py -- Phase A  
| +-- cpsat_v2_timetable.py -- Phase B  
| +-- decomposition_spectral_v2.py  
| +-- metaheuristics.py -- LNS / SA / TS / ILS  
| +-- classroom_assignment.py -- Phase 4  
+-- schedule/ -- legacy single-file prototypes  
+-- webui/  
| +-- start.bat -- launcher Windows
```

```
| +-- start.sh -- launcher Linux/macOS
| +-- stop.sh -- stop helper
| +-- backend/ -- FastAPI app
| +-- frontend/ -- SvelteKit
| +-- data/timetable.db -- SQLite
| +-- docs/ -- guide brevi user-facing
+-- proposals/ -- design notes + benchmarks
+-- *.pdf -- reference papers
```

## 3.4 Lifecycle del backend

webui/backend/main.py definisce un lifespan che chiama `init_db()` allo startup. Esso fa:

1. `Base.metadata.create_all(bind=engine)` – crea le tabelle mancanti.
2. `_apply_lightweight_migrations()` – ALTER TABLE idempotenti per i campi aggiunti in versioni successive (esempio: `school_classes.curriculum_id`, `teachers.last_name/first_name/nickname` ecc.). Ogni ALTER è guardato da `PRAGMA table_info` per non duplicare.

## 4. Modello dati

Tutto vive in `webui/backend/models.py`. Le tabelle si raggruppano in cinque famiglie tematiche.

### 4.1 Famiglie di tabelle

#### Anagrafiche

`subjects`, `teachers`, `school_classes`, `classrooms`, `curricula`, `students`, `study_groups`.

#### Tag (M-N)

`classroom_tags` + `classroom_tag_assignments`, `student_tags` + `student_tag_assignments`.  
Nome unique lower-cased, descrizione opzionale, cancellazione cascata.

#### Assegnazione

`class_subjects`, `curriculum_subject_hours`, `teacher_subjects`, `subject_group_weights`,  
`assignments`.

#### Disponibilità / vincoli

`teacher_unavailability`, `class_unavailability`, `classroom_unavailability`, `logical_unavailabilitie`,  
`curriculum_logical_constraints`, `classroom_subject_preferences`, `teacher_classroom_preferences`,  
`coteaching_rules`.

#### Soluzioni e scheduling

`solutions`, `lessons`, `day_counts`.

#### Coverage e Run

`absences`, `substitute_assignments`, `runs`, `run_logs`, `app_state`.

#### Viste salvate

`saved_views`: query DSL + sort multi-livello salvati per una entità lista, esposti dal dropdown  
"Viste salvate" del componente *SortableQueryableList*. UNIQUE (`entity`, `name`).

### 4.2 Diagramma relazioni

### 4.3 Migrazioni idempotenti

Pattern (in `db.py`):

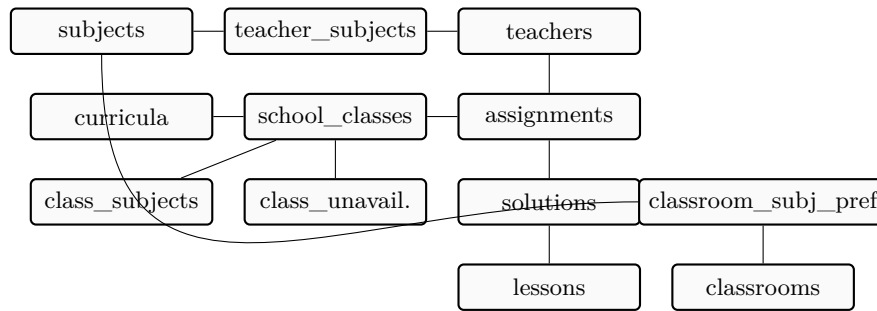


Figura 4.1: Relazioni rilevanti del modello dati (semplificato).

```

if insp.has_table("school_classes") \
    and not has_column("school_classes", "curriculum_id"):
    conn.execute(text(
        "ALTER TABLE school_classes ADD COLUMN curriculum_id INTEGER"
    ))

```

Ogni ALTER è guardato da una check su PRAGMA table\_info. Quando aggiungi una colonna nuova, segui lo stesso pattern: il DB preesistente sopravvive senza interventi manuali.

## 4.4 Pickle dell'engine

engine\_io.py converte fra DB e tre forme pickle:

```

school = {
    'profile': str,
    'classes': [{name, year, section, curriculum, subjects:{subj:ore}}],
    'teachers': [{name, group, max_hours, free_day,
                  weights:{subj:int}}],
    'cconcorsopersubject': {subj: {group: weight}},
    'curriculum_scores': {curriculum: int},
    'curricula': [...],
    'students': [...],
    'groups': [...],
}
profs = {
    teacher_name: {
        'classi': {class_name: {subject: {'ore': N}}},
        'glibero': [d1, d2, d3]
    }
}
solution = {(prof, class, subj, day, hour): 0|1}

```

## 5. Vincoli

### 5.1 I 5 stati / 4 kind

Il sistema usa una tassonomia uniforme di *5 stati* sia per le matrici di disponibilità (cella per cella) sia per le preferenze di aula. I vincoli logici disgiuntivi (DNF) hanno *4 kind* (ALLOWED non si applica qui).

| Stato     | Colore       | Semantica                                                                                                   |
|-----------|--------------|-------------------------------------------------------------------------------------------------------------|
| ALLOWED   | verde chiaro | default (solo grid materia↔aula e docente↔aula); nessuna contribuzione all'obiettivo                        |
| SOFT      | giallo       | penalità positiva quando lo slot è usato (+ <b>penalty</b> aggiunto all'obiettivo, che è minimizzato)       |
| HARD      | rosso        | il solver deve evitare questo slot, oppure il vincolo logico DEVE essere soddisfatto                        |
| PREFERITO | blu          | bonus negativo quando lo slot è usato / il vincolo logico è soddisfatto (- <b>bonus</b> riduce l'obiettivo) |
| ENFORCED  | verde scuro  | il solver DEVE far accadere questo slot / soddisfare il vincolo con presence required                       |

### 5.2 Matrici di disponibilità 5-stati

Il ciclo cliccando una cella è:

`free → soft → hard → preferred → enforced → free`

Le celle gialle e blu mostrano un input numerico per editare la penalità (positiva sul giallo, negativa sul blu, sign-clamp automatico al salvataggio).

**Sincronizzazione free\_day.** Quando il docente ha `free_day` settato, tutte e 6 le ore di quel giorno appaiono HARD nella matrice automaticamente. Se l'utente edita la matrice manualmente e tutte le ore di un giorno diventano HARD, il backend deduce il `free_day`; viceversa cambiare il `free_day` sostituisce le auto-cells.

**Mapping CP-SAT.** In `optimization.py::_availability_constraints` ogni tabella si traduce in: `<entity>_hard` (set di tuple, forbidden assignment), `<entity>_soft` (dict tuple→penalty signed), `<entity>_enforced` (set di tuple, presence required nel solver).

### 5.3 Vincoli logici (DNF)

#### 5.3.1 Sintassi

```

expr := or_expr
or_expr := and_expr ('OR' and_expr)*
and_expr := not_expr ('AND' not_expr)*
not_expr := ('NOT' | 'mai') not_expr | atom
atom := slot | predicate | '(' expr ')'
slot := <giorno><ora>
predicate:= <kind> ':' <name> ('@' <giorno> <ora>)?
kind := aula | materia | classe | gruppo | docente | prof
giorno := lun|mar|mer|gio|ven|sab (anche lunedì|...|sabato,
        monday|...|saturday)
ora := 1..6 (ordinale, 1=08:00) | 8..13 (assoluta)

```

Sono accettati & (AND), | (OR), ! (NOT). mai è alias di NOT.

### 5.3.2 Predicate atoms

```

aula:LabFisica -- l'entita' usa l'aula a qualunque slot
aula:LabFisica@mar -- il martedì a qualunque ora
aula:LabFisica@mar4 -- martedì 4a ora (= 11:00)
materia:Religione@lun8 -- lezione di religione lunedì alle 8
classe:1A_Scientifico -- riferimento alla classe
gruppo:IRC -- riferimento al gruppo

```

I literal predicate vengono parsati e persistiti correttamente. Il solver li tratta otticamente per ora; l'integrazione strong con la fase classroom assignment è pianificata.

### 5.3.3 Esempi pratici

```

% Per docenti
lun8 AND lun9
(lun8 AND lun9) OR (mar8 AND mar9)
gio11 OR gio12 OR gio13
NOT (mer3 AND mer4)
mai aula:LabFisica@mar
mai materia:Religione@lun8

% Per classi
mer4 AND mer5 AND mer6
mai aula:Palestra@gio
aula:LabInformatica@mar OR aula:LabInformatica@gio

% Per aule
gio4 AND gio5 AND gio6
mai materia:Religione
gruppo:IRC OR gruppo:Alternativa

```

### 5.3.4 Mapping CP-SAT

Per ogni regola con DNF [clause1, clause2, ...]:

- HARD: il solver impone almeno una clausola fully active.
- SOFT: se nessuna clausola è fully active, paga +soft\_penalty.
- PREFERITO: se almeno una clausola è fully active, ottiene +soft\_penalty (negativo, quindi reduce dell'obiettivo).

- **ENFORCED**: come **HARD** ma con *presence required* – almeno uno degli slot della DNF deve coincidere con un'ora di lezione attiva per quell'entità.

## 5.4 Vincoli **HARD** a toggle (per classe)

In `school_classes` ci sono 7 boolean strutturali: `hard_entry_at_8`, `hard_exit_after_12`, `hard_no_holes`, `hard_dual_math`, `hard_dual_italian`, `hard_motorie_pairs`, `hard_max_6_per_day`. Il soft `soft_minimize_sixth_weight` regola il peso della 6a ora.

## 5.5 Conflict detection

Il tab *Vincoli* ha un bottone "Cerca conflitti" (GET `/api/monitor/conflicts`) che segnala:

- **matrix\_hard\_enforced** – stessa cella marcata sia **HARD** sia **ENFORCED**.
- **enforced\_on\_free\_day** – cella **ENFORCED** del docente nel suo **free\_day**.
- **logical\_unsatisfiable** – vincolo logico **HARD/ENFORCED** la cui DNF non è soddisfacibile dato l'insieme degli **HARD** slot dell'owner.

# 6. Workflow di ottimizzazione

## 6.1 Schema delle fasi

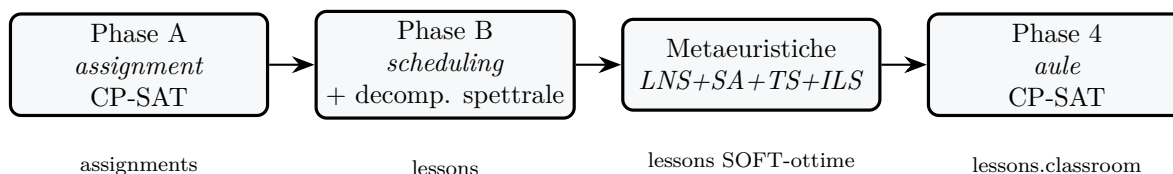


Figura 6.1: Pipeline a 4 fasi.

## 6.2 Phase A: assegnazione (cattedre)

**Modulo.** `experiments/cpsat_v2_assignment.py`.

**Lancio.** `POST /api/optimize/assignment` con `{time_limit_s, workers, log}`.

**Input.** L'insieme di `(class, subject, hours_per_week)` letto da `class_subjects` e/o `curriculum_subjects` + le abilitazioni docente in `teacher_subjects` + la classe-di-concorso preference in `subject_group_weights`.

**Output.** La tabella `assignments` (`teacher x class x subject` con `hours`). Problema di covering: ogni `(class, subject)` riceve un docente abilitato, rispettando max-hours per docente, classi-di-concorso e `teacher_compatible_classes`.

## 6.3 Phase B: scheduling

**Modulo.** `experiments/cpsat_v2_timetable.py` + `experiments/decomposition_spectral_v2.py`.

**Decomposizione spettrale.** Per istanze grandi, il problema CP-SAT integrale è intrattabile. La pipeline calcola un grafo di co-occupazione fra docenti, ne fa un'embedding spettrale via Laplaciano normalizzato e usa K-means su quell'embedding per partizionare in  $k$  cluster *loosely-coupled*. Ogni cluster diventa una sub-istanza CP-SAT indipendente; un secondo step "bridges" risolve solo i bordi; un terzo step "ricucitura" fa full-CPSAT con tutto il problema partendo dalla soluzione stitched.

## 6.4 Metaeuristiche

**Modulo.** `experiments/metaheuristics.py`. Cascata di:

- **LNS** – destroy-and-repair su finestre (`teacher`, `day`) o (`class`, `day`); freeze del 60-70% delle lezioni e CP-SAT-resolve del resto.
- **SA** (Simulated Annealing) – single-swap accettati con Metropolis a  $T$  decrescente con fattore  $\alpha$ .
- **TS** (Tabu Search) – best-improvement local search con tabu list (default size 80).
- **ILS** (Iterated Local Search) – alterna  $n$  cicli di TS con *kick* perturbativi (random 4-opt fra giorni).

Tutti contribuiscono al SOFT score: holes per teacher, isolated 1- and 5-hour days, weekly distribution, dual-hour pairs, no 6th hour, banded preferences su materie. Il SOFT include la contribuzione delle matrici di disponibilità SOFT/PREFERITO e dei logical SOFT/PREFERITO.

## 6.5 Phase 4: assegnazione aule

**Modulo.** `experiments/classroom_assignment.py`. Input: la tabella `lessons` (già temporalmente fissata) + le classroom rules (`kind`, `capacity`, `multi_class`, preferenze materia↔aula, classe↔aula, docente↔aula). Output: `lessons.classroom_name` popolato.

## 6.6 Drag-and-drop con preview live

Trascinando una lezione su un altro slot la UI fa due chiamate:

1. POST `/api/schedule/move-preview` – il backend simula la mossa per tutte le 36 (`day`, `hour`) e per ognuna restituisce:
  - `ok` con `delta_soft`  $\leq 0$  – mossa accettabile (verde se `delta`  $< 0$ , verde chiaro se `delta`  $= 0$ ).
  - `soft_worse` – mossa accettabile ma peggiora il SOFT (giallo).
  - `hard_violation` – vincolo HARD violato (rosso).
  - `noop` – slot di origine.
2. L'utente droppa, la UI invia PUT `/api/schedule/move-lesson` che valida e persiste.

### 6.6.1 Room-follows-lesson

Il move-preview NON considera l'aula nel calcolo HARD. Se la nuova posizione è compatibile con docente / classe ma l'aula attuale è già occupata o HARD-unavailable in destination, il backend:

1. Sposta la lezione.
2. Lascia l'aula libera sul nuovo slot solo se non c'è conflitto.
3. Se c'è conflitto, rimuove l'aula dalla lezione spostata e risponde con `room_cleared=true`, `cleared_room=<nome>`.
4. La UI apre un Modal che spiega cosa è successo e chiede di scegliere una nuova aula dal dropdown (verde libera / rosso occupata).

## 6.7 Slot picker matriciale (Monitor)

Nel tab *Monitor*, espandendo una cattedra si vedono le singole lezioni; cliccando il bottone *Giorno/Ora* si apre un modal con una matrice 6x6 dove ogni cella è colorata in base alla disponibilità:

- verde: free (docente e classe liberi)
- rosso: HARD (docente o classe già impegnati)
- ambra: aula occupata da un'altra lezione
- azzurro: slot attuale

Click su una cella libera fa un dry-run via PUT `/api/monitor/event/{aid}/lesson/{lid}`. Se ci sono conflitti, apre un secondo Modal con 3 opzioni: **Annulla**, **Disassegna le lezioni in conflitto**, **Disassegna e ottimizza dopo**.

# 7. Guida UI

## 7.1 Funzionalità trasversali

### 7.1.1 Query DSL e sort multi-livello

Ogni pagina lista ha:

- barra Query (DSL: =, !=, <, <=, >, >=, contains, startswith, endswith, in [...], logica AND / OR, parentesi);
- sort multi-livello (max 4 livelli);
- pannello Help con la lista dei campi.

### 7.1.2 Multi-select

Sulle pagine Docenti / Classi / Aule la lista supporta:

- click semplice: single-select (replace);
- Ctrl+click: toggle;
- Shift+click: range;
- bottone "Vincolo collettivo" che apre il *BulkApplyModal*.

### 7.1.3 Excel/CSV import

Ogni pagina lista ha "Importa Excel/CSV" + "Template". Lo stesso endpoint POST `/api/import/{entity}` gestisce le 7 entità (teachers, subjects, classes, classrooms, curricula, students, groups). Modalità: **upsert** (default), **append**, **replace**. Header alias italiani / inglesi accettati.

### 7.1.4 Esportazione liste (xlsx / csv)

Sopra ogni tabella, accanto a "Reset query" / "Reset sort", trovi 3 bottoni di esportazione:

- **xlsx**: la *vista corrente* (solo le righe filtrate dalla query DSL, nell'ordine determinato dal sort multi-livello) come file `.xlsx` con header colorato e auto-fit colonne.
- **csv**: stesso contenuto, formato CSV UTF-8 con BOM (compatibile con Excel italiano).
- **tutto**: ignora query e sort, esporta tutte le righe dell'entità.

Filename: `<entita>_<YYYYMMDD>_<HHMMSS>.<ext>`. I parametri `?format=xlsx|csv` sono accettati da tutti gli endpoint di lista, quindi gli stessi export sono disponibili anche via curl.

### 7.1.5 Viste salvate

Sopra ogni tabella, accanto ai bottoni di Reset e Export, trovi un dropdown "Viste salvate..." e i bottoni "Salva vista" / "x":

- **Salva vista:** chiede un nome e memorizza la query DSL + il sort multi-livello come `SavedView(entity, name, dsl_query, sort_levels)`. La vista è globale.
- **Dropdown "Viste salvate...":** applica una vista (sostituisce query + sort + ricarica la lista).
- **x:** elimina la vista correntemente selezionata.

Endpoint: `/api/saved-views?entity=<entity>` (GET / POST / PUT {id} / DELETE {id}). Tabella: `saved_views`.

### 7.1.6 Tag (aule e studenti)

Sistema many-to-many parallelo per aule e studenti.

**Tag aule:** tabella `classroom_tags` + join `classroom_tag_assignments`. UI: multi-select autocomplete nella scheda aula, bottone "Gestisci tag" per CRUD globale, colonna "Tag" con pill colorate (un colore stabile per nome). Il generatore mock tagga automaticamente le aule per kind (`lab_fisica` → [`lab`, `fisica`, `scienze`], `palestra` → [`palestra`, `motorie`], ecc.) e propaga il curriculum dalla home class (`3B_scientifico` → tag `scientifico`).

DSL: `"matematica"` in `l.classroom.tags` (vincoli) o `has_tag(matematica)` (query lista). I tag sconosciuti vengono creati al volo al salvataggio.

**Tag studenti:** tabella `student_tags` + join `student_tag_assignments`. Casi d'uso: BES, DSA, `debito_matematica_4`, `pcto_ditta_x`, `studente_atleta`. UI analoga a quella delle aule. Pannello "Precompila da tag" nel modal *Gruppi* che aggiunge in massa gli studenti che matchano `any_of` o `all_of` su una lista di tag.

**Importante:** i tag NON sostituiscono i gruppi. I gruppi restano l'unità operativa di scheduling, i tag sono metadato passivo per filtrare/raggruppare.

### 7.1.7 Import bulk dedicato (/import)

Pagina dedicata raggiungibile dalla nav. Layout a due colonne: configurazione (entità, modalità, "Scarica template") + file da importare (drop area + selettore file). Il template xlsx ha 3 fogli:

- **Istruzioni:** tabella **Campo** | Hint con descrizione di ogni colonna + note generali (sinonimi italiani, righe vuote ignorate, etc.);
- **Esempi:** una riga di esempio pre-popolata;
- **Dati:** foglio vuoto da compilare. L'importer legge per default questo foglio.

Campi importabili già integrati con le feature recenti: `tags` su `students` e `classrooms`, `graduatoria_score` / `required_free_days_count` / `preferred_free_days_json` su `teachers`, e `max_hours_per_day` su `classes`.

### 7.1.8 Import / Export DB (Dashboard)

Card prominente nella prima riga della Dashboard. Funzioni:

- **Esporta DB completo:** scarica un `.zip` contenente `database.db` (raw SQLite), `tables/<t>.csv` per ogni tabella e `metadata.json` (schema\_version + sha256).

- **Solo schema:** omette i dati di scheduling – utile come template per inizializzare un'altra scuola.
- **Import:** selettore file .zip + conferma esplicita. Una copia `timetable.db.pre_import_backup` viene salvata prima dello swap.
- **Snapshot:** timestampati in `webui/data/snapshots/`, lista con bottoni Ripristina / Elimina.

Endpoint: `/api/dashboard/export-db`, `/api/dashboard/import-db`, `/api/dashboard/snapshot/{create}`, `DELETE /api/dashboard/snapshot/{f}`.

## 7.2 Tab per tab

### Dashboard

Importa profilo (5 size) con 3 checkbox per i pool (curricula, aule, studenti); genera scuola di test; stato corrente con 7 card-numero; *RunLogPanel* streaming.

### Docenti, Classi, Indirizzi, Studenti, Gruppi, Materie, Aule, Compresenze

CRUD entità con modal di edit. Vedere `ui_guide.md` per i campi specifici di ognuna.

### Cattedre

Layout a card: una card per classe, lista materie+docente+ore. Bottone "cambia": apre modal con dropdown filtrato per materia, ogni opzione mostra `Cognome Nome - assigned/max [SFORA: +Xh] / [pieno]`. Bottone "Warnings cattedre": pannello con docenti SFORA / sotto / vuoti / ok.

### Orario

4 viste (per classe, per docente, per aula, per slot), ognuna con toggle matrice/lista. Drag-and-drop con preview live, dropdown aule colorate (verde libera / rosso occupata).

### Assenze e supplenze

Vista settimanale 6×6. Click su intestazione giorno: modal gestione assenze. Click su cella rossa: modal con classi scoperte (sinistra) + docenti disponibili (destra), drag-drop per assegnare supplenti.

### Monitor

Lista *eventi* (uno per Assignment). Espansione lezione-per-lezione con bottone Giorno/Ora che apre slot picker matriciale (vedere §5.6).

### Vincoli

Lista flat di tutti i vincoli editabili con pill colorate per livello. Bottone "Cerca conflitti": pannello con conflitti rilevati.

### Workflow

Pulsanti per lanciare le 4 fasi separatamente o "Pipeline completa". SSE log streaming.

## 8. Tecniche di ottimizzazione avanzate

Oltre alle metaeuristiche classiche (LNS / SA / TS / ILS) il progetto include cinque tecniche aggiuntive. Vedere `docs/optimization_strategies.md` per la specifica completa.

### 8.1 Adaptive Large Neighborhood Search (ALNS)

`experiments/alns.py`. Sei destroy operator (`random_window`, `day_cluster`, `worst_fit_day`, `teacher_day`, `classroom_day`, `single_class_week`) e tre repair operator (`cp_sat_window`, `greedy_by_soft`, `bfs_fill_back`) selezionati con roulette wheel sopra exponentially-decayed scores; acceptance SA-like.

### 8.2 Variable Neighborhood Search (VNS)

`experiments/vns.py`. Cicla quattro vicinati di dimensione crescente (1-swap, 2-swap, 3-chain, k-opt con  $k \in [4, 6]$ ). Ad ogni miglioramento riparte dal vicinato 1; termina quando un'intera passata 1→4 non trova nulla.

### 8.3 Hall's theorem pre-check

`experiments/diagnostics/hall_check.py`. Diagnostico sincrono pre-Phase-A. Tre controlli:

1. per (class, subject) coverage,
2. subject supply vs demand,
3. Hall sampling random-subset con sottoinsiemi esclusivi.

Esposto in *tre* punti UI: bottone inline in PhaseACard, riga in AdvancedTechniquesCard, sezione del tab Statistiche.

### 8.4 Column Generation (Dantzig-Wolfe)

`experiments/column_generation.py`. Master LP via `scipy.linprog` HiGHS; sotto-problema per docente. Skeleton single-iteration per scuole molto grandi ( $> 200$  classi). Default OFF nella pipeline.

### 8.5 Lagrangian Relaxation

`experiments/lagrangian.py`. Rilassa i ponti inter-cluster e li dualizza con multipliers  $\lambda_b$ ; aggiornamento via subgradient ascent  $\lambda_{k+1} = \max(0, \lambda_k + \alpha_k g_k)$  con  $\alpha_k = \alpha_0 / (1 + k)$ . Per-iteration refinement via SA.

## 9. DSL generico per i vincoli

Linguaggio compatto, dichiarativo, totalmente componibile per esprimere vincoli HARD/SOFT/PREFERITO/ENFORCED su qualunque combinazione logica di predicati sul modello dei dati. Implementato in `webui/backend/utils/general_dsl.py` (parser ricorsivo discendente + AST + evaluator post-hoc, ~250 LOC, no dipendenze esterne). Esposto via `POST /api/constraints/general`, validato live da `POST /api/constraints/general/validate`, accessibile dal modal “+ Nuovo vincolo DSL” del tab Vincoli.

### 9.1 Filosofia

Prima del DSL generico il sistema aveva quattro sub-DSL specializzati (query, logical, objective, introspettivo). Stessa grammatica implementata 4 volte. Il DSL generico unifica tutto sotto una sola grammatica con quattro “sapori” di compilazione (LIST, LOGIC, OBJECTIVE, EVAL): *un parser, molti compilatori*. La sintassi di `forall x in S where Filter: Body` e dei predicati atomici (`==`, `!=`, `<`, `in`, ...) è identica in tutti i contesti; cambia solo il backend di traduzione.

Il DSL è interpretato *post-hoc*: parsea l’espressione e la valuta DOPO che la pipeline (Phase A, Phase B, metaeuristiche) ha prodotto una soluzione candidata. Le violazioni HARD sono rilevate dal Feasibility Check; le SOFT alimentano lo score globale.

### 9.2 Grammatica BNF

```
expr ::= iff_expr
iff_expr ::= implies_expr ( "<=>" implies_expr ) *
implies_expr ::= or_expr ( "=>" or_expr ) *
or_expr ::= and_expr ( ("OR"|"or") and_expr ) *
and_expr ::= not_expr ( ("AND"|"and") not_expr ) *
not_expr ::= ("NOT"|"not") not_expr | atom

atom ::= quant_expr | count_expr | comparison
      | call | bool_lit | "(" expr ")"

quant_expr ::= ("forall"|"exists") IDENT "in" source
            ["where" or_expr] ":" expr

count_expr ::= "count" IDENT "in" source ["where" or_expr]
            ( ":" comparison | op value )

source ::= IDENT -- literal name
        | IDENT ( "." IDENT ) + -- path (runtime)

comparison ::= value ( op value
                    | "in" "[" value ( "," value ) * "]"
                    | "in" path
                    | "not_in" "[" value ( "," value ) * "]"
```

```

        | "not_in" path )

op ::= "==" | "!=" | "<" | "<=" | ">" | ">="
value ::= IDENT ( "." IDENT )* | NUM | STRING
        | BOOL | function_call
function_call ::= IDENT "(" [arg_list] ")"
arg_list ::= arg ("," arg)*
arg ::= IDENT "==" value | value

```

Precedenze (dal più basso al più alto): <=>, ==>, OR, AND, NOT, confronti, . (member access). Le parentesi sovrascrivono.

## 9.3 Sorgenti iterabili

| Sorgente    | Tipo elemento                               |
|-------------|---------------------------------------------|
| lessons     | lezioni della soluzione attiva              |
| assignments | cattedre (Assignment rows)                  |
| teachers    | docenti (name, group, max_hours, subject)   |
| classes     | classi (name, year, curriculum, n_students) |
| classrooms  | aule (name, kind, type, tags)               |
| students    | studenti (con tags[] e groups[])            |
| subjects    | materie                                     |
| curricula   | indirizzi di studio                         |
| groups      | gruppi articolati                           |
| slots       | le 36 (day, hour) della settimana           |
| days        | i 6 giorni (Lun..Sab; index 1..6)           |
| hours       | le 6 ore (8..13)                            |

**Sorgente-path:** dalla v0.5 puoi scrivere `exists g in s.groups: g.name == "X"` dove `s.groups` e' un attributo che restituisce una lista; il parser accetta qualunque path puntato dopo `in`.

## 9.4 Galleria di esempi

Ogni esempio sotto e' stato validato live contro il parser corrente; copia-incolla nell'editor del modal "+ Nuovo vincolo DSL" e funziona.

### 9.4.1 Vincoli docente (10 esempi)

1. Borghi non insegna mai in 1A alla 6a ora:

```

forall l in lessons where l.teacher == Borghi
                        and l.class == 1A:
    l.hour != 13

```

2. Borghi: max 4 ore/giorno totali:

```

forall d in days:
    count l in lessons where l.teacher == Borghi
                        and l.day == d.index: l <= 4

```

3. Lab fisica: ogni docente di Fisica esattamente 1 ora a settimana in lab\_fisica:

```

forall t in teachers where t.subject == "Fisica":
    count l in lessons where l.teacher == t
                        and l.classroom.type == "lab_fisica": l == 1

```

4. Tetto cattedra Inglese: max 18h/settimana:

```
forall t in teachers where t.subject == "Inglese":
    count l in lessons where l.teacher == t: l <= 18
```

5. Rossi non lavora il sabato (HARD):

```
forall l in lessons where l.teacher == Rossi
    and l.day == 6:
    false
```

6. Rossi: al più 1 sesta ora/settimana:

```
count l in lessons where l.teacher == Rossi
    and l.hour == 13: l <= 1
```

7. Tutti A050: max 5 ore/giorno:

```
forall t in teachers where t.group == "A050":
    forall d in days:
        count l in lessons where l.teacher == t
            and l.day == d.index: l <= 5
```

8. Coupling Rossi 3A  $\Rightarrow$  Bianchi 3B:

```
(exists l in lessons:
    l.teacher == Rossi and l.class == 3A)
=> (exists l in lessons:
    l.teacher == Bianchi and l.class == 3B)
```

9. Rossi mai in palestra:

```
forall l in lessons where l.teacher == Rossi:
    l.classroom.type != "palestra"
```

10. Tutti i prof di mate: almeno una lezione di mattina:

```
forall t in teachers where t.subject == "Matematica":
    exists l in lessons where l.teacher == t:
        l.hour <= 9
```

### 9.4.2 Vincoli classe (5 esempi)

1. La 1A non lavora il sabato:

```
forall l in lessons where l.class == 1A: l.day != 6
```

2. Le prime: max 5 ore/giorno:

```
forall c in classes where c.year == 1:
    forall d in days:
        count l in lessons where l.class == c.name
            and l.day == d.index: l <= 5
```

3. Quarte/quinte: due ore CONSECUTIVE di mate:

```
forall c in classes where c.year >= 4:
  exists s1 in slots: exists s2 in slots:
    consecutive(s1, s2)
    and lesson(class==c.name, day==s1.day,
               hour==s1.hour, subject==Matematica)
    and lesson(class==c.name, day==s2.day,
               hour==s2.hour, subject==Matematica)
```

4. Linguistico anno 1: niente sabato:

```
forall l in lessons
  where l.class.curriculum == Linguistico
  and l.day == 6: false
```

5. 1A: religione solo nelle prime 4 ore:

```
forall l in lessons where l.class == 1A
  and l.subject == Religione:
  l.hour <= 11
```

### 9.4.3 Vincoli aula (5 esempi)

1. Lab Fisica 1: una sola lezione per slot:

```
forall s in slots:
  count l in lessons
    where l.classroom == "Lab Fisica 1"
    and l.day == s.day and l.hour == s.hour:
  l <= 1
```

2. Matematica solo in aule taggate “matematica”:

```
forall l in lessons where l.subject == Matematica:
  "matematica" in l.classroom.tags
```

3. Storia delle terze: aula con proiettore:

```
forall l in lessons where l.subject == Storia
  and l.class.year >= 3:
  "proiettore" in l.classroom.tags
```

4. La palestra ospita SOLO Scienze motorie:

```
forall l in lessons
  where l.classroom.type == "palestra":
  l.subject == Scienzemotorie
```

5. Lab chimica: max 2 lezioni concorrenti:

```
forall s in slots:
  count l in lessons
    where l.classroom.type == "lab_chimica"
    and l.day == s.day and l.hour == s.hour:
  l <= 2
```

### 9.4.4 Vincoli materia (3 esempi)

1. Mate distribuita: max 2 ore/giorno per classe:

```
forall c in classes:
  forall d in days:
    count l in lessons
      where l.class == c.name
        and l.subject == Matematica
        and l.day == d.index: l <= 2
```

2. Mate preferibilmente entro la 5a ora:

```
forall l in lessons where l.subject == Matematica:
  l.hour <= 12
```

3. Educazione fisica solo in palestra:

```
forall l in lessons
  where l.subject == Educazione fisica:
    l.classroom.type == "palestra"
```

### 9.4.5 Vincoli gruppo / studenti (3 esempi)

1. Studenti BES devono essere in un gruppo Sostegno (uso di `exists g in s.groups`):

```
forall s in students where "BES" in s.tags:
  exists g in s.groups: g.name == "Sostegno"
```

2. Studenti con debito mate quarto → gruppo Recupero:

```
forall s in students
  where "debito_matematica_4" in s.tags:
    exists g in s.groups:
      g.name == "Recupero Matematica"
```

3. Studenti BES: nessuna lezione il sabato:

```
forall l in lessons:
  forall s in students
    where l.class == s.class
      and "BES" in s.tags:
        l.day != 6
```

### 9.4.6 Vincoli relazionali / coupling (4 esempi)

1. Rossi in Aula 12: solo Matematica:

```
forall l in lessons where l.teacher == Rossi
  and l.classroom == "Aula 12":
  l.subject == Matematica
```

2. Se Rossi in 3A allora deve usare lab fisica almeno una volta:

```
(exists l in lessons:
  l.teacher == Rossi and l.class == 3A)
=>
(exists l2 in lessons:
  l2.teacher == Rossi and l2.class == 3A
  and l2.classroom.type == "lab_fisica")
```

3. Inglese alle prime: max 3 ore/sett per (docente,classe):

```
forall t in teachers where t.subject == "Inglese":  
  forall c in classes where c.year == 1:  
    count l in lessons  
      where l.teacher == t  
        and l.class == c.name: l <= 3
```

4. Religione alle prime: mai in 1a ora:

```
forall l in lessons where l.subject == Religione  
  and l.class.year == 1:  
  l.hour != 8
```

## 9.5 Errori comuni

- **Atteso COLON, trovato OP (=)** – usa == nei confronti dopo **where** (= singolo e' lecito, ma confonde il parser quando appare in contesti ambigui).
- **sorgente sconosciuta 'foo'** – sorgente non in `_VALID_SOURCES` e non e' un path raggiungibile via env. Controlla l'ortografia.
- **oltre 1000000 atomi: probabile esplosione combinatoria** – quantificatori troppo annidati. Sposta filtri nel **where** più esterno per ridurre il set; oppure splitta il vincolo in più vincoli salvati separatamente.

Vedere `docs/general_dsl.md` per la guida completa con sezioni 1–12 (galleria di 30+ esempi commentati, troubleshooting esteso, reference tabellare di tutti gli attributi per ciascuna entità).

# 10. Diagnostica statistica

Il tab `/diagnostics` aggrega tutte le analisi sul modello e sulla soluzione attiva. Vedere `docs/diagnostics.md`. Visualizzazioni con Apache ECharts (lazy-loaded), tema *pitantum* (palette indaco / oro / avorio / terra-di-siena).

## 10.1 Sensitivity Monte Carlo

Genera  $N$  perturbazioni feasible (sequenze di mosse atomiche random accettate solo se HARD soddisfatto) e calcola statistiche SOFT. Coefficiente di variazione  $< 0.05 \Rightarrow$  soluzione vicina all'ottimo locale.

## 10.2 Analisi bipartito

Tre metriche networkx: densita' del grafo classe $\times$ docente, modularita' Newman-Girvan greedy, top-K betweenness centrality.

## 10.3 Correlazioni e regressioni

Tre regressioni statsmodels: OLS `load_vs_gaps`, OLS `subjects_vs_soft`, Logit `saturation_vs_sixth`. Output con coefficienti, p-values,  $R^2$  (o pseudo- $R^2$ ) e interpretazione testuale auto-generata.

## 10.4 Distribuzioni

Cinque distribuzioni (carichi docenti, classi per giorno, materia $\times$ slot heatmap, occupancy slot, KS-test, chi-quadro) + goodness-of-fit.

## 10.5 Run telemetry

Tabella `run_telemetry` (alembic `a848420325b3`). I moduli solver pushano sample via `utils.telemetry.collect` (phase). Il run detail page (`/runs/[id]`) mostra il grafico objective vs tempo (multi-linea per phase) + statistiche per stage + export JSON.

# 11. API REST

Tutti gli endpoint sono sotto `/api/`. Backend espone ~118 route. Lista non esaustiva delle più rilevanti.

## 11.1 Pattern CRUD

Ogni risorsa CRUD ha la stessa forma:

| Verbo  | Path                                        | Descrizione      |
|--------|---------------------------------------------|------------------|
| GET    | <code>/api/&lt;entity&gt;</code>            | lista (q + sort) |
| GET    | <code>/api/&lt;entity&gt;/&lt;id&gt;</code> | dettaglio        |
| POST   | <code>/api/&lt;entity&gt;</code>            | create           |
| PUT    | <code>/api/&lt;entity&gt;/&lt;id&gt;</code> | replace          |
| DELETE | <code>/api/&lt;entity&gt;/&lt;id&gt;</code> | delete           |

## 11.2 Esempi curl

```
# stato dataset
curl http://127.0.0.1:8000/api/dataset/state

# import small con tutti i pool
curl -X POST -H "Content-Type: application/json" \
  -d '{"profile":"small","use_optimized":true,
      "import_curricula":true,"import_classrooms":true,
      "import_students":true}' \
  http://127.0.0.1:8000/api/dataset/import-profile

# vincolo logico HARD su un docente
curl -X POST -H "Content-Type: application/json" \
  -d '{"expression":"mai aula:LabFisica@mar","kind":"hard"}' \
  http://127.0.0.1:8000/api/teachers/15/logical-unavailabilities

# bulk dry-run su 3 classi
curl -X POST -H "Content-Type: application/json" \
  -d '{"entity_ids":[1,2,3],"action":"add_logical",
      "payload":{"expression":"lun8 AND lun9","is_hard":true,
                  "soft_penalty":100},
      "on_conflict":"dry_run"}' \
  http://127.0.0.1:8000/api/bulk/classes/dry-run

# coverage settimana
curl 'http://127.0.0.1:8000/api/coverage/week?week_start=2026-04-27'

# event lessons (monitor)
curl http://127.0.0.1:8000/api/monitor/event/1/lessons
```

## 11.3 Riferimento gruppi di endpoint

- **Anagrafiche:** /api/teachers, /classes, /classrooms, /subjects, /curricula, /students, /groups.
- **Cattedre:** /api/assignments + teachers-for-subject + loads.
- **Vincoli logici:** /api/{teachers,classes,classrooms}/{id}/logical-unavailabilities, /api/curricula/{cid}/logical-constraints.
- **Bulk ops:** /api/bulk/{entity}/dry-run|apply.
- **Schedule:** /api/schedule/by-class|by-teacher|by-room|by-slot, move-lesson, move-preview, lesson/{lid}/classroom.
- **Coverage:** /api/coverage/week|cell, /api/absences, /api/substitutions.
- **Optimization:** /api/optimize/assignment|phase-b|lms|sa|ts|ils|full|classroom-assignment.
- **Monitor:** /api/monitor/events|summary|constraints|conflicts, event/{aid}/lessons|lesson/{id}.
- **Import:** /api/import/{entity}, /api/import/{entity}/template.

## 12. Estendere il sistema

### 12.1 Aggiungere una nuova tabella + CRUD

1. Modello in `webui/backend/models.py`.
2. Pydantic in `schemas.py` (`XxxIn`, `XxxOut`).
3. Router in `routers/xxx.py`. Copia un router esistente come template.
4. Includi il router in `main.py`: `app.include_router(xxx.router)`.
5. Adapter DSL in `utils/list_query.py`.
6. Frontend: nuova route `webui/frontend/src/routes/xxx/+page.svelte`, riusando *SortableQueryableList*.
7. Voce in `+layout.svelte` per la nav.

### 12.2 Migrazione idempotente

Aggiorna `db.py::_apply_lightweight_migrations`. Pattern:

```
if insp.has_table("my_table") \
    and not has_column("my_table", "new_field"):
    conn.execute(text(
        "ALTER TABLE my_table ADD COLUMN new_field VARCHAR(64)"
    ))
```

### 12.3 Aggiungere un nuovo tipo di vincolo

**Per-cella.** Copia il pattern di `TeacherUnavailability`: `unique (entity_id, day, hour)`, columns `state/penalty/reason`. Aggiorna `models.py`, `schemas.py`, router, `optimization.py::_availability_constraints`, `routers/monitor.py::_build_constraints` (per il tab Vincoli), `routers/bulk.py::ENTITY_UNAV_MODEL` se vuoi supportare bulk.

**Logico.** Aggiungi `entity_type` a `LogicalUnavailability`; aggiorna `routers/logical.py::ENTITY_MODEL` e `ENTITY_TYPE`; modifica `routers/monitor.py::_build_constraints`; aggiungi il branch nel solver.

### 12.4 Aggiungere un nuovo step di ottimizzazione

```
def my_new_step(...):
    params = dict(...)
    run_id = create_run("my_step", "Nome leggibile", profile, params)

    def target(rid: int):
        # carica dati, lancia solver, persiste
        update_run(rid, progress=0.5)
        ...
        update_run(rid, solution_id=sid, obj_value=v, metrics=m)
        update_run(rid, progress=1.0)

    start_thread(run_id, target)
    return run_id
```

Esponi via `routers/optimize.py` con un endpoint POST. Frontend in `/optimize` per il bottone di lancio + log streaming via *RunLogPanel* (SSE).

## 12.5 Testing

Non c'è una test suite formale. Per smoke-testare end-to-end:

- Backend boot pulito: `uvicorn backend.main:app`.
- Live curl: vedere §7.2.
- Frontend `vite build` deve passare clean.
- Migrazione idempotente: rilancia su DB esistente per confermare zero data loss.

# A. Launcher

## A.1 Windows: `start.bat`

Apri due finestre cmd separate. Pre-flight:

- `npm` reachable (forza `nodejs` nel PATH);
- `venv` esiste in `backend/.venv`;
- `backend/main.py` e `frontend/package.json` presenti;
- se `node_modules` manca, lancia `npm install`;
- avvisa se 8000 / 5173 sono già occupate.

I path sono relativi a `%~dp0`, quindi lo script funziona ovunque venga messo.

## A.2 Linux / macOS: `start.sh` + `stop.sh`

Default background: log in `webui/logs/`, PID in `webui/logs/pids`. Flag `-f` / `-foreground` per Ctrl+C cleanup. Auto-crea `venv` e fa `npm install` se mancano. Hint per `apt`, `dnf`, `pacman`, `brew` se `python3` / `npm` non sono nel PATH.

`stop.sh` legge `logs/pids`, manda SIGTERM con fallback SIGKILL dopo 3s, killa anche i child via `pkill -P` e ha un fallback per ammazzare qualunque processo ancora in ascolto su 8000 / 5173 via `lsof`.

## B. Repository GitHub

URL: <https://github.com/gborghi/timetable>.

Repository privato. La landing page contiene il README di alto livello con quickstart, tour della UI, sintassi vincoli, architettura, repository layout.

## C. Reference

- Pisinger, "Where are the hard knapsack problems?", 2005.
- Kohl, Karisch, Patriksson, "Very Large-Scale Neighborhood Search Techniques", 2002.
- Burke, Kingston, Pepper, "A standard data format for timetabling instances", 2008 (S1877050919318800).
- Ahuja, Magnanti, Orlin, "Network Flows", 1993.